

```
import matplotlib as mpl
import matplotlib.font_manager as fm
import os
```

```
# — Registrar Arial
```

```
def _registrar_arial():
    fontes = [f.name for f in fm.fontManager.ttflist]
    if "Arial" in fontes:
        return
    candidatos = [
        r"C:\Windows\Fonts\arial.ttf",
        "/Library/Fonts/Arial.ttf",
        "/usr/share/fonts/truetype/msttcorefonts/Arial.ttf",
        os.path.join(os.path.dirname(__file__), "Arial.ttf"),
    ]
    for caminho in candidatos:
        if os.path.exists(caminho):
            fm.fontManager.addfont(caminho)
            return
    import urllib.request
    dest = os.path.join(os.path.expanduser("~"), "Arial.ttf")
    if not os.path.exists(dest):
        url = "https://github.com/matomo-org/travis-scripts/raw/master/fonts/Arial.ttf"
        urllib.request.urlretrieve(url, dest)
    fm.fontManager.addfont(dest)
```

```
_registrar_arial()
mpl.rcParams["font.family"] = "Arial"
```

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import matplotlib.lines as mlines
from matplotlib.patches import FancyBboxPatch
from scipy.stats import gaussian_kde
import statsmodels.api as sm
import statsmodels.formula.api as smf
```

```
# — Diretório de saída
```

```
OUTPUT_DIR = os.path.dirname(os.path.abspath(__file__))
os.makedirs(OUTPUT_DIR, exist_ok=True)
```

```
OUT_PDF = os.path.join(OUTPUT_DIR, "raincloud_idade_desfecho.pdf")
OUT_SVG = os.path.join(OUTPUT_DIR, "raincloud_idade_desfecho.svg")
```

```
# — Reprodutibilidade
```

```
rng = np.random.default_rng(42)
```

```
# — Dados observados
```

```
grupos = {
    "Discharge": dict(n=429, med=26.0, q1=2.0, q3=56.0, xbar=29.9, dp=28.3, vmin=0, vmax=94),
    "Death": dict(n=274, med=65.0, q1=54.0, q3=76.0, xbar=63.3, dp=16.9, vmin=1, vmax=97),
    "Overall": dict(n=703, med=50.0, q1=8.0, q3=67.5, xbar=42.9, dp=29.4, vmin=0, vmax=97),
}
```

```

CORES = {
    "Discharge": "#2a9d8f",
    "Death": "#e07a2a",
    "Overall": "#7c6fc0",
}

# — Simula distribuições calibradas nos quartis observados —————
def simular(g, seed):
    rng2 = np.random.default_rng(seed)
    vmin, vmax = g["vmin"], g["vmax"]
    med_norm = (g["med"] - vmin) / (vmax - vmin + 1e-9)
    a = max(0.5, med_norm * 4)
    b = max(0.5, (1 - med_norm) * 4)
    s = rng2.beta(a, b, g["n"]) * (vmax - vmin) + vmin
    return np.clip(s, vmin, vmax)

amostras = {k: simular(v, i * 7) for i, (k, v) in enumerate(grupos.items())}

# — Regressão Binomial Negativa: Idade ~ Desfecho —————
idade_disc = amostras["Discharge"].astype(int)
idade_death = amostras["Death"].astype(int)

df_nb = pd.DataFrame({
    "idade": np.concatenate([idade_disc, idade_death]),
    "desfecho": np.array([0] * len(idade_disc) + [1] * len(idade_death)),
})
df_nb["idade"] = df_nb["idade"].clip(lower=1)

modelo_nb = smf.negativebinomial("idade ~ desfecho", data=df_nb).fit(dis=False)

beta = modelo_nb.params["desfecho"]
ci_low = modelo_nb.conf_int().loc["desfecho", 0]
ci_hi = modelo_nb.conf_int().loc["desfecho", 1]
p_val = modelo_nb.pvalues["desfecho"]

if p_val < 0.001:
    p_str = "p < 0.001"
    sig = "****"
elif p_val < 0.01:
    p_str = f"p = {p_val:.3f}"
    sig = "***"
elif p_val < 0.05:
    p_str = f"p = {p_val:.3f}"
    sig = "**"
else:
    p_str = f"p = {p_val:.3f}"
    sig = "ns"

print("\n— Regressão Binomial Negativa: Idade ~ Desfecho —")
print(f"β = {beta:.4f}")
print(f"IC 95% = [{ci_low:.4f}, {ci_hi:.4f}]")
print(f"{p_str} ({sig})")
print(modelo_nb.summary())

# — Delta da mediana (Death – Discharge) —————
delta_med = grupos["Death"]["med"] - grupos["Discharge"]["med"] # +39

```

— Figura

```
fig, ax = plt.subplots(figsize=(15, 10))
fig.patch.set_facecolor("white")
ax.set_facecolor("#fafafa")

ys = np.arange(len(grupos))
labels = list(grupos.keys())
JITTER_N = 80
KDE_MAX = 0.40

for i, (nome, g) in enumerate(grupos.items()):
    cor = CORES[nome]
    amostra = amostras[nome]
    y0 = ys[i]

    # 1. Nuvem KDE
    kde = gaussian_kde(amostra, bw_method=0.25)
    x_kde = np.linspace(0, 100, 300)
    dens = kde(x_kde)
    dens_n = dens / dens.max() * KDE_MAX

    ax.fill_between(x_kde, y0, y0 + dens_n, color=cor, alpha=0.50, zorder=2)
    ax.plot(x_kde, y0 + dens_n, color=cor, lw=1.4, alpha=0.75, zorder=3)

    # 2. Jitter
    n_jit = min(g["n"], JITTER_N)
    idx = rng.choice(len(amostra), n_jit, replace=False)
    jitter = rng.uniform(-0.12, 0.0, n_jit)
    ax.scatter(amostra[idx], y0 + jitter - 0.05,
               color=cor, alpha=0.38, s=16, linewidths=5, zorder=3)

    # 3. Box plot compacto
    BH = 0.10
    by = y0 - 0.30

    ax.hlines(by, g["vmin"], g["q1"], colors=cor, lw=2.4, ls=(0, (4, 3)), zorder=4)
    ax.hlines(by, g["q3"], g["vmax"], colors=cor, lw=2.4, ls=(0, (4, 3)), zorder=4)
    for xw in [g["vmin"], g["vmax"]]:
        ax.vlines(xw, by - BH * 0.7, by + BH * 0.7, colors=cor, lw=2.4, zorder=4)

    caixa = FancyBboxPatch(
        (g["q1"], by - BH), g["q3"] - g["q1"], 2 * BH,
        boxstyle="round,pad=0.3",
        linewidth=1.8, edgecolor=cor,
        facecolor=cor + "33", zorder=5,
    )
    ax.add_patch(caixa)

    ax.vlines(g["med"], by - BH, by + BH, colors=cor, lw=3.0, zorder=6)
    ax.vlines(g["xbar"], by - BH * 0.7, by + BH * 0.7, colors="gray", lw=2.4,
              ls=(0, (5, 4)), zorder=6)

    # 4. Anotações estatísticas
    ax.text(g["med"] + 1, by + BH - 0.05,
            f'Md={g["med"]:.0f}', fontsize=15,
            color=cor, fontweight="bold", va="bottom", fontfamily="Arial")

    ax.text(g["xbar"] + 1, by - BH - 0.03,
            f'x̄={g["xbar"]:.1f}', fontsize=15,
```

```

        color="gray", va="top", fontfamily="Arial")

ax.text(-1, by,
        f"Q1={g['q1']:.0f}",
        fontsize=15, color=cor, ha="right", va="center", fontfamily="Arial")

ax.text(103, by,
        f"Q3={g['q3']:.0f}",
        fontsize=15, color=cor, ha="left", va="center", fontfamily="Arial")

# — Delta da mediana: seta Discharge → Death


---


y_disc_box = ys[0] - 0.30 # y central do box Discharge
y_death_box = ys[1] - 0.30 # y central do box Death
x_med_disc = grupos["Discharge"]["med"] # 26
x_med_death = grupos["Death"]["med"] # 65
x_arrow = (x_med_disc + x_med_death) / 2 # ponto médio horizontal

# Linha vertical tracejada ligando as duas medianas (no eixo x de cada)
# — seta dupla no eixo x, entre x=26 e x=65, posicionada entre os dois grupos
y_arrow_mid = (y_disc_box + y_death_box) / 2 # entre os dois grupos

# Seta horizontal dupla no nível y_arrow_mid
ax.annotate(
    "",
    xy = (x_med_death, y_arrow_mid),
    xytext = (x_med_disc, y_arrow_mid),
    arrowprops=dict(
        arrowstyle = "<->",
        color = "#555",
        lw = 1.8,
        shrinkA = 0,
        shrinkB = 0,
    ),
    zorder=7,
)

# Linhas verticais tracejadas das medianas até y_arrow_mid
for x_m, y_grp in [(x_med_disc, y_disc_box), (x_med_death, y_death_box)]:
    ax.plot(
        [x_m, x_m],
        [y_grp, y_arrow_mid],
        color="#555", lw=1.4, ls=(0, (4, 3)), zorder=6,
    )

# Rótulo Δ Md
sign_str = f"+{delta_med:.0f}" if delta_med >= 0 else f"{delta_med:.0f}"
ax.text(
    (x_med_disc + x_med_death) / 2.1,
    y_arrow_mid + 0.08,
    f"Δ Md = {sign_str} yrs",
    fontsize=14, color="#555", fontweight="bold",
    ha="center", va="bottom", fontfamily="Arial",
    bbox=dict(boxstyle="round,pad=0.25", facecolor="white",
              edgecolor="#bbb", alpha=0.85),
    zorder=8,
)

# — Barra de significância com resultado NB (Discharge ↔ Death) —————
x_bar = 102

```

```

ax.annotate("", xy=(x_bar, y_death_box), xytext=(x_bar, y_disc_box),
            arrowprops=dict(arrowstyle="-", color="#444", lw=1.0))
ax.hlines([y_disc_box, y_death_box], x_bar - 0.5, x_bar, colors="#444", lw=1.0)

nb_txt = (
    f"NB Regression\n"
    f" $\beta = \{beta:.3f\}$ \n"
    f"95% CI [ $\{ci_low:.3f\}$ ,  $\{ci_hi:.3f\}]$ \n"
    f" $\{p\_str\}$   $\{sig\}$ "
)
ax.text(x_bar + 0.8, (y_disc_box + y_death_box) / 2,
        nb_txt,
        fontsize=13, color="#333", fontweight="bold",
        va="center", ha="left", fontfamily="Arial",
        linespacing=1.6)

# — Eixo Y: labels customizados (nome + n abaixo) —————
# Remove os ytick labels padrão e desenha manualmente dois textos por grupo
ax.set_yticks(ys)
ax.set_yticklabels([""] * len(labels)) # apaga labels padrão

# Largura do texto no eixo — usa transform para posicionar à esquerda do plot
LABEL_X = -7 # coordenada em unidades de dados (ax.set_xlim começa em -5)

for i, nome in enumerate(labels):
    cor = CORES[nome]
    y0 = ys[i]

    # Linha 1: nome do grupo (bold, maior)
    ax.text(
        LABEL_X, y0 + 0.08,
        nome,
        fontsize=20, fontweight="bold", color=cor,
        ha="right", va="center", fontfamily="Arial",
        transform=ax.transData,
    )

    # Linha 2: n= (menor, mesmo alinhamento)
    ax.text(
        LABEL_X, y0 - 0.09,
        f"n = {grupos[nome]['n']}",
        fontsize=18, fontweight="bold", color="#888",
        ha="right", va="center", fontfamily="Arial",
        transform=ax.transData,
    )

# — Eixos —————
ax.set_xlim(-5, 125)
ax.set_ylim(-0.75, len(grupos) - 0.3)
ax.set_xlabel("Age (years)", fontsize=18, color="#555", fontfamily="Arial")
ax.xaxis.set_tick_params(labelsize=10, labelcolor="#666")
ax.xaxis.set_major_locator(plt.MultipleLocator(10))
ax.grid(axis="x", color="#ddd", linewidth=0.7, linestyle="--", alpha=0.8, zorder=0)
ax.spines[["top", "right"]].set_visible(False)
ax.spines[["left", "bottom"]].set_color("#ccc")
ax.tick_params(axis="y", length=0) # remove tracinhos do eixo y

for lbl in ax.get_xticklabels():
    lbl.set_fontfamily("Arial")

```

— Legenda

```
patches = [mpatches.Patch(facecolor=CORES[k] + "55",
                           edgecolor=CORES[k], lw=1.5, label=k)
            for k in labels]
l_med = mlines.Line2D([], [], color="gray", lw=2.5, label="Median")
l_mean = mlines.Line2D([], [], color="gray", lw=1.4, ls=(0, (5, 4)), label="Mean ( $\bar{x}$ ")
l_jit = mlines.Line2D([], [], marker="o", color="gray",
                      markersize=5, lw=0, alpha=0.5,
                      label="Individual data (jitter, n≤80)")

leg = ax.legend(handles=[*patches, l_med, l_mean, l_jit],
                framealpha=0.88, edgecolor="#ddd",
                loc="upper right", ncol=2,
                prop={"family": "Arial", "size": 12})
```

— Exportação

```
fig.savefig(OUT_PDF, dpi=300, bbox_inches="tight", facecolor="white")
fig.savefig(OUT_SVG, bbox_inches="tight", facecolor="white")
plt.show()

print("\nExportado:")
print(f" {OUT_PDF}")
print(f" {OUT_SVG}")
```